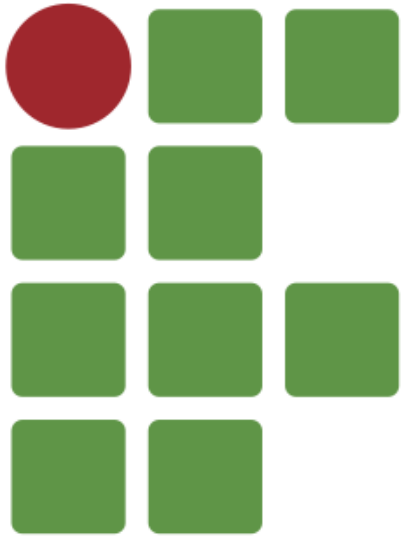


INSTITUTO FEDERAL
Brasília

ARQUITETURA DE COMPUTADORES

Prof. MSc. Thiago Neves
IFB Campus Taguatinga
08/11/2026

AULA 09

Assembler MIPS – Parte 3

if-then-else

Em C e MIPS

Jumps condicionais

beq $t1, $t2, label # Se $t1 == $t2, então vá para o label

bne $t1, $t2, label **#** Se $t1 != $t2, então vá para o label

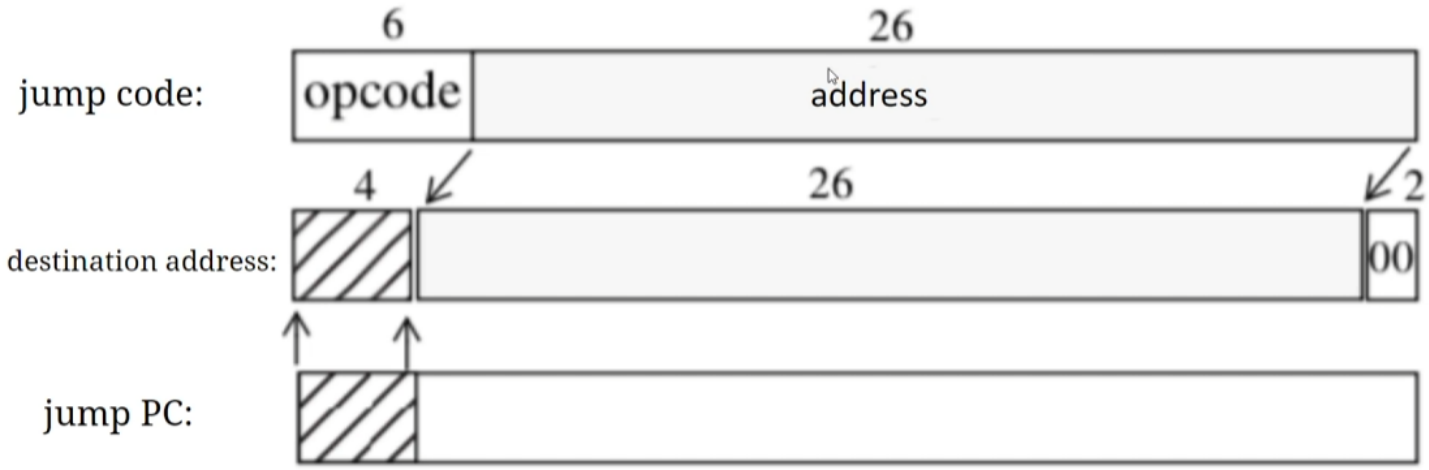
Jumps condicionais

macros blt/bgt/bge/ble/bltu/bgtu/bgeu/bleu	
blt rs, rt, label	If (rs < rt) jump to label
bgt rs, rt, label	If (rs > rt) jump to label
bge rs, rt, label	If (rs >= rt) jump to label
ble rs, rt, label	If (rs <= rt) jump to label

Jumps Incondicionais

j label, vá para o label

- Endereço de destino = 26bits
- CPU completa os demais 6 bits
  - 4 bits copiados do PC.
  - 2 bits menos significativos são 0.



Jumps Incondicionais

j target → PC = target

jr rs → PC = rs

jal target → PC = target ; $ra = PC_{up}

jalr rs, rd → PC = rs ; $rd = PC_{up}

Exemplo:

```
main:
    # some code here
    jal function
    # more code here

function:
    # function code here
    jr $ra
```

if-then-else Statement

Exemplo em C:

```
if (a>=b)
    d = a;
else
    d = b;
```

Exemplo em MIPS:

```
1    blt $t0, $t1, else
2    move $t3, $t0
3    b endif
4  else:
5    move $t3, $t1
6  endif:
```

if-then-else Statement (AND)

Exemplo em C:

```
if (a >= b && a < c )
   d = a;
else
   d = b;
```

Exemplo em MIPS:

```
1     blt $t0, $t1, else
2     bge $t0, $t2, else
3     move $t3, $t0
4     b endif
5  else:
6     move $t3, $t1
7  endif:
```

if-then-else Statement (OR)

Exemplo em C:

```
if (a >= b || a < c )
  d = a;
else
  d = b;
```

Exemplo em MIPS:

```
1    bge $t0, $t1, then
2    bge $t0, $t2, else
3  then:
4    move $t3, $t0
5    b fisi
6  else:
7    move $t3, $t1
8  endif:
```

if-then-else Statement (OR-AND-OR)

Exemplo em C:

```
if ((A||B)&&(C||D)) {
    S;
}
```

Pseudocódigo:

```
    If A TRUE jump to AND
    If B FALSE jump to ENDIF
AND:
    If C TRUE jump to THEN
    If D FALSE jump to ENDIF
THEN:
    S
ENDIF:
```

Loops

Em C e MIPS

While – Divisão Inteira

Exemplo em C:

```
q = 0;
while (dd >= dr) {
   dd = dd - dr;
   q++;
}
```

Exemplo em MIPS:

```
move $t3, $zero

while:
   blt $t1, $t2, endwhile
   subu $t1, $t1, $t2
   addiu $t3, $t3, 1
   b while

endwhile:
```

While loop – Divisão Inteira (Otimização)

Exemplo em C:

```
q = 0;
while (dd >= dr) {
   dd = dd - dr;
   q++;
}
```

Exemplo em MIPS:

```
move $t3, $zero

blt $t1, $t2, endwhile

while:
   subu $t1, $t1, $t2
   addiu $t3, $t3, 1
   bge $t1, $t2, while

endwhile:
```

For loop

Exemplo em C:

```
for (i = 0; i < 5; ++i)  {
    a = a + i ;
}
```

Exemplo em MIPS:

```
# Initialize i to 0
li $t1, 0
# Load 5 into $t3
li $t3, 5
loop:
# Check if i is greater or equal to 5
   bge $t1, $t3, exit
# Add i to a
   addu $t2, $t2, $t1
# Increment i
   addiu $t1, $t1, 1
   j loop
```

Do-While loop

Exemplo em C:

```
do {
  k = k + b;
  b = b - 1;
}
while (b > 0 && k <10);
```

Exemplo em MIPS:

```
loop:
   add  $t0, $t0, $t1    # k = k + b
   addi $t1, $t1, -1     # b = b - 1

   blez $t1, end         # Se b <= 0, termina

   addi $t2, $zero, 10   # t2 = 10
   slt  $t3, $t0, $t2    # t3 = 1 se k < 10

   bne  $t3, $zero, loop # Se k < 10, repete

end:
```

Subrotinas

Subrotinas

Subprograma com determinados parâmetros, que pode retornar um resultado e permite ser chamado a partir de vários pontos do programa.

Ação/Procedimento, Função (C), Método (Java).

Projeto top-down (de cima para baixo), reutilização de código, programação modular etc.

Subrotinas

Em C:

```
#include <stdio.h>
// Declaração da função
int sum(int a, int b);

int main(void) {
   int x;
   x = sum(2, 3);
   x = x + 1;
   printf("Resultado: %d\n", x);

   return 0;
}

// Implementação da função
int sum(int a, int b) {
   return a + b;
}
```

Em MIPS:

```
main:
   ...
   jal sum          # Jump and link: $ra = ret, PC = sum

ret:
   ...

sum:
   ...
   jr $ra           # Jump register: PC = $ra
```

Regra 1: Parâmetros e Resultados

Parâmetros: $a0, $a1, $a2, $a3 (valores de ponto flutuante utilizam $f12 e $f14). Em ordem.

Resultados: $v0 ($f0 para valores de ponto flutuante).

Observação 1: Realizar extensão de sinal (sign extension) ou preenchimento com zeros (zero extension) quando o tipo possuir menos de 32 bits.

Observação 2: Outros casos não são permitidos (tipos long, estruturas struct etc.).

Regra 1: Parâmetros e Resultados

Em C:

```c
#include <stdio.h>

// Função que recebe dois inteiros e retorna a soma
int sum(int a, int b) {
   return a + b;
}

int main(void) {
   int x;
   x = sum(2, 3);
   x = x + 1;
   printf("Resultado: %d\n", x);
   return 0;
}
```

Em MIPS:

```
main:
   ...
   li $a0, 2
   li $a1, 3
   jal sum

   # we do something with $v0
   ...

sum:
   addu $v0, $a0, $a1
   jr $ra
```

Regra 1: Parâmetros e Resultados

Em C:

```
int v1[10];

int main() {
   init(v1);
}

int init(int v2[]) {
   int i;

   for (i = 0; i < 10; i++) {
      v2[i] = 0;
   }
}
```

Em MIPS:

```
main:
   ...
   la $a0, v1              # a0 = &v1
   jal init
init:
   li $t0, 0               # i = 0
   li $t1, 10
for:
   bge $t0, $t1, endfor    # jump to endfor if i >= 10
   sll $t2, $t0, 2         # t2 = 4i
   addu $t2, $t2, $a0      # t2 = &v1 + 4i → address of v1[i]
   sw $zero, 0($t2)        # v1[i] = 0
   addiu $t0, $t0, 1
   b for
endfor:
   jr $ra
```

Regra 2: Variáveis Locais

Escolha livre: $t0...$t9, $s0...$s7, $v0...$v1

Números de ponto flutuante de precisão simples: $f0...$f31

Exceções (serão armazenadas na pilha – stack):

1. **Vetores e matrizes.**
2. **Quando uma variável v aparece como &v (endereço da variável).**
3. **Quando não houver mais registradores disponíveis.**

Regra 2: Variáveis Locais

Em C:

```
int func() {
    int a;
    a = 1;
    return a;
}
```

Em MIPS:

```
func:
    li $v0, 1
    jr $ra
```

Regra 2: Variáveis Locais

A pilha e o bloco de ativação (quadro de pilha – stack frame)

- Subrotinas: É necessário salvar os dados na memória e liberar o último bloco armazenado.
- Estrutura LIFO (Last In, First Out): O último elemento a entrar é o primeiro a sair.
- Ponteiro da pilha: É representado pelo registrador $sp.
- Alocação de espaço: O espaço é reservado no início e liberado ao final. As instruções PUSH e POP não são utilizadas.

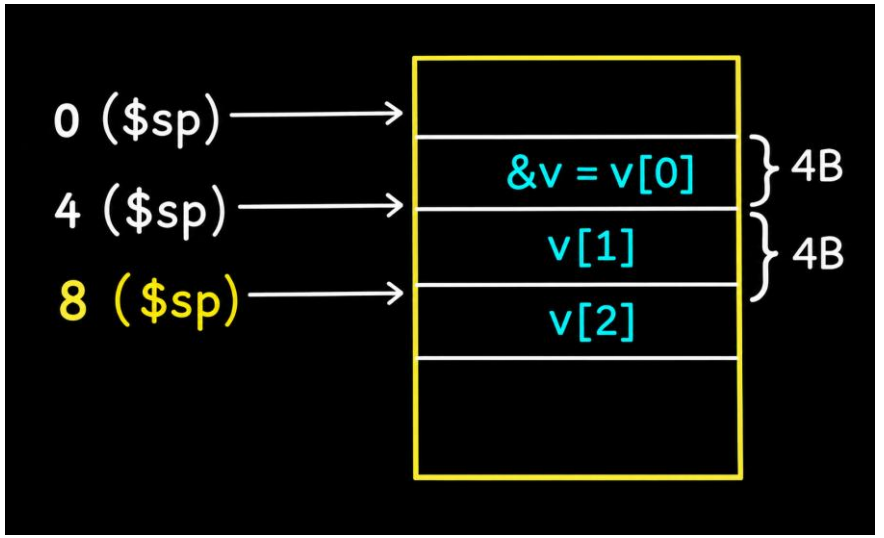
Regra 2: Variáveis Locais

Em C:

```
int f() {
   int i;
   int vec[10];
   i = vec[2];
   return i;
}
```

Em MIPS:

```
f:
   addiu $sp, $sp, -40
   lw    $v0, 8($sp)
   addiu $sp, $sp, 40
   jr    $ra
```



Regra 3: Bloco de Ativação

Variáveis em ordem

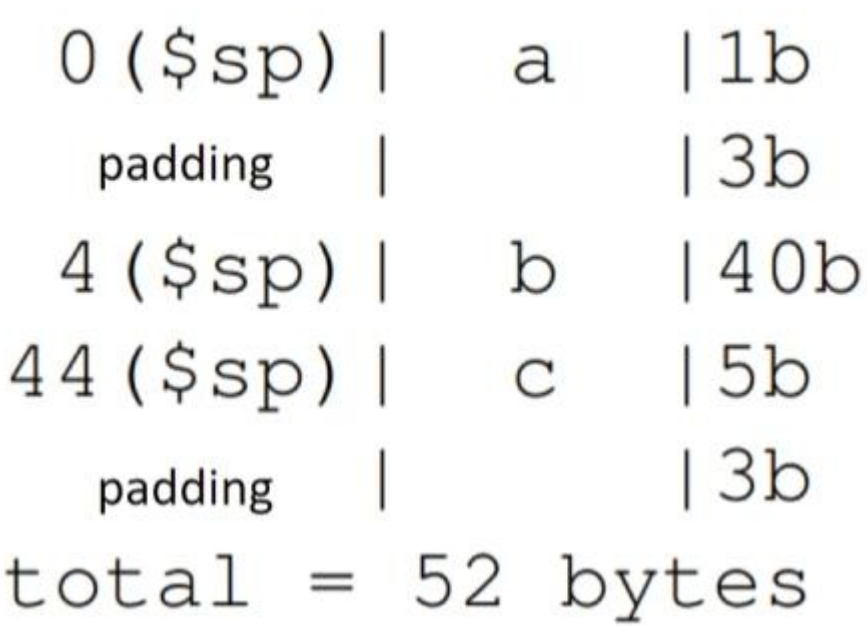
- Alinhadas como se estivessem na seção .data.
- Tamanho total múltiplo de 4 bytes.

O bloco de ativação (activation block) é um bloco contíguo de memória alocado na pilha (stack), que armazena as variáveis locais de uma subrotina, bem como outras informações relacionadas à execução da subrotina, como o endereço de retorno e o ponteiro do quadro de pilha (frame pointer).

O bloco de ativação é criado em tempo de execução, quando a subrotina é chamada, e destruído quando a subrotina retorna.

Regra 3: Bloco de Ativação

```
1  int f1() {
2      int i;
3      char a;
4      int b[10];
5      char c[5]
6      ...
7      ... &a ...
8      return i;
9  }
```



Regra 3: Bloco de Ativação – Subrotinas de múltiplos níveis

```
int f1(int a) {
    int b = 10;
 ---f2()---
    return a+b;
}
```

a, b and $ra are at risk

Regra 4: Preservar Contexto

Uma subrotina deve preservar os valores originais dos registradores $s0 a $s7.

Os valores necessários desses registradores devem ser protegidos (salvos pelo procedimento chamado).

Conforme a convenção de chamada, os valores originais serão armazenados na pilha (stack), após as variáveis locais.

Todo esse procedimento será realizado no início e no final da subrotina, e não no local onde ocorre a chamada (em estruturas condicionais, laços de repetição etc.).

Regra 4: Preservar Contexto

```
int f1(int a) {
    int b = 10;
 ---f2()---
    return a+b;
}
```

a, b and $ra are at risk

```
f1:
   addiu $sp, $sp, -12
   sw $s0, 0($sp)          # OLD VALUE $s0
   sw $s1, 4($sp)          # OLD VALUE $s1
   sw $ra, 8($sp)          # OLD VALUE $ra
   move $s0, $a0           # $s0 = VALUE TO PRESERVE
   li $s1, 10              # $s1 = VALUE TO PRESERVE

   jal f2
   lw $s0, 0($sp)
   lw $s1, 4($sp)
   lw $ra, 8($sp)
   addiu $sp, $sp, 12

   addu $v0, $s0, $s1

   jr $ra
```

Regra 4: Preservar Contexto

Situation 1: x = f() vs. *x = f()

```
1  int f1 () {
2      int x = 1;
3      x = f2 ();
4      return x;
5  }
```

(no need to preserve x)

```
1  void f1 (int *x) {
2      *x = f2 ();
3  }
```

Ponteiro pode estar em risco!

(need to preserve x → since f2() may change the context of pointer x)

Regra 4: Preservar Contexto

```
1  int f1(int c) {
2          int s = 0;
3      for (i = 0; i<c; i++)
4          s = s + f2();
5      return s;
6  }
```

(i, c and s must be saved)

Exercícios Respondidos

Exercício 1

Traduzir a declaração de variáveis em C para MIPS.

C

```
char aa             = -5;
short bb            = -344;
long long cc        = -3;
unsigned char dd = 0xA0;
int ee              = 5799;
short  ff           = -1;
```

Exercício 1 - Resposta

C

```
char aa             = -5;
short bb            = -344;
long long cc        = -3;
unsigned char dd    = 0xA0;
int ee              = 5799;
short  ff           = -1;
```

```
.data
aa:  .byte -5          # char aa = -5
     .align 1
bb:  .half -344        # short bb = -344
     .align 3
cc:  .word -1, -3      # long long cc = -3 (64 bits)
dd:  .byte 0xA0        # unsigned char dd = 0xA0
     .align 2
ee:  .word 5799        # int ee = 5799
     .align 1
ff:  .half -1          # short ff = -1
```

Exercício 1 - Resposta

Por que cc: .word –1, –3?

O valor –3 em 64 bits (complemento de dois) é:

`0xFFFFFFFFFFFFFFFD`

Dividido em dois words de 32 bits:

Parte alta (32 bits)	Parte baixa (32 bits)
`0xFFFFFFFF`	`0xFFFFFFFD`
(= –1)	(= –3)

Layout na memória (exemplo – início em `0x1001000`)

Endereço	Conteúdo (hex)	Variável	Descrição
0x10010000	FB	aa	–5 (1 byte)
0x10010001	(padding)	–	(alinhamento para 2 bytes)
0x10010002	FE A8	bb	–344 (2 bytes)
0x10010004	(padding)	–	(alinhamento para 8 bytes)
0x10010008	FF FF FF FF	cc (parte alta)	–1 (32 bits)
0x1001000C	FF FF FF FD	cc (parte baixa)	–3 (32 bits)
0x10010100	A0	dd	0xA0 (1 byte)
0x10010101	(padding)	–	(alinhamento para 4 bytes)
0x10010104	00 00 16 A7	ee	5799 (4 bytes)
0x10010108	FF FF	ff	–1 (2 bytes)

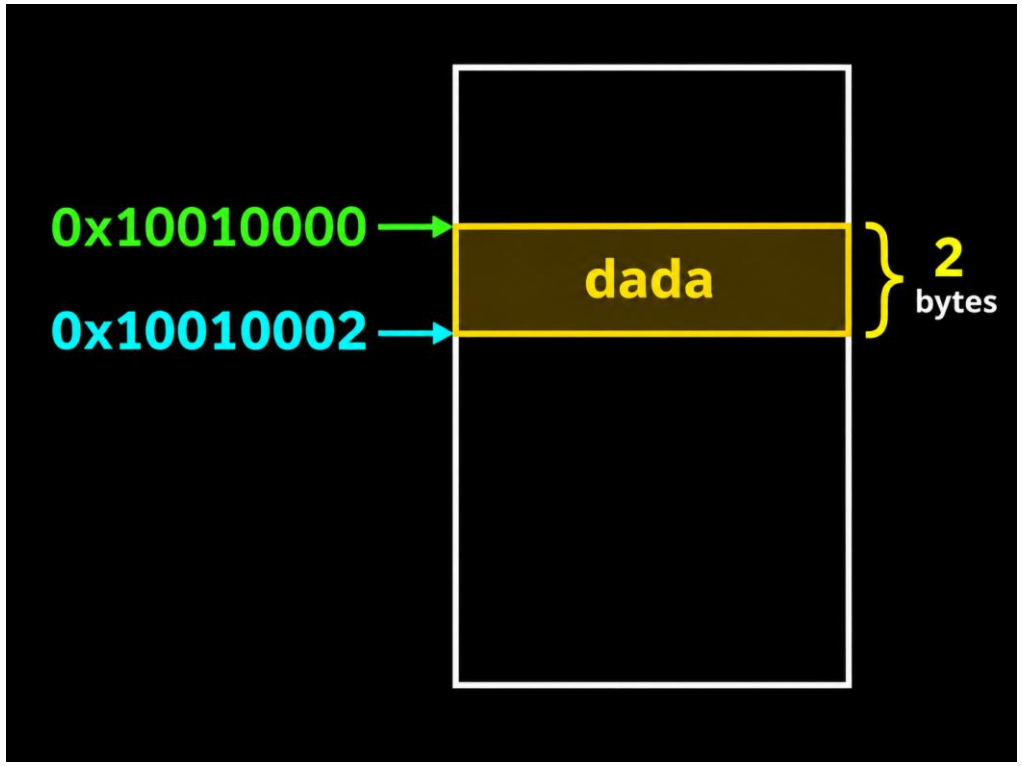
Exercício 2

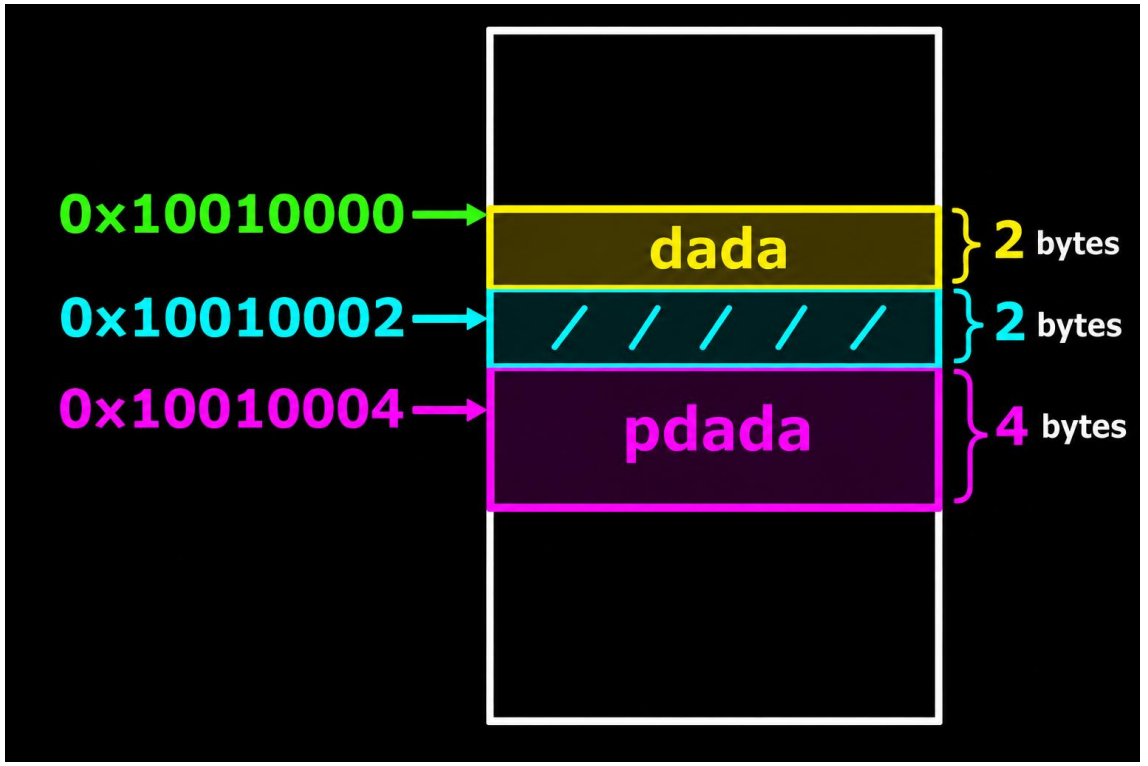
Dada a seguinte declaração de dados em Assembly MIPS (um ponteiro inicializado com o endereço de outra variável global) e supondo que as variáveis estejam armazenadas na memória a partir do endereço 0x10010000, escreva o valor hexadecimal de cada uma das expressões a seguir, escritas em linguagem C:

```
        .data
dada:   .half 3
pdada:  .word dada
```

&pdada		&dada	
pdada		dada	
*pdada			

Exercício 2 - Resposta





Exercício 2 - Resposta

```
        .data
dada:   .half 3
pdada:  .word dada
```

&pdada	0x10010004	&dada	0x10010000
pdada	0x10010000	dada	0x0003
*pdada	0x0003		

Exercício 3

Traduzir de C para Assembler MIPS. A variável temporária deve ser salva no registrador $s0.

```
int A[3] = {3, 5, 7};
int *pointer = 0;
void main() {
        int temp;
        pointer = &A[2];
        temp = *pointer + 2;
        temp = *(pointer-2) + temp;
        A[1] = temp;
        print_integer(temp);
        // main returns to startup code
}
```

Exercício 3 - Resposta

```
.data
A:      .word 3, 5, 7      # int A[3] = {3, 5, 7}
pointer: .word 0           # int *pointer = 0;
temp:    .word 0           # int temp;

.text

main:
   # pointer = &A[2];
   la   $t0, A             # address of A
   addi $t1, $t0, 8        # address of A[2]
   sw   $t1, pointer       # store pointer to A[2]

   # temp = *pointer + 2;
   lw   $s0, ($t1)         # load A[2] into temp
   addi $s0, $s0, 2        # add 2 to temp

# temp = *(pointer-2) + temp;
   subi $t2, $t1, 8        # pointer - 2
   lw $t3, ($t2)           # load A[0] into $t3
   add $s0, $s0, $t3       # add A[0] to temp

   # A[1] = temp;
   addi $t4, $t0, 4        # address of A[1]
   sw $s0, ($t4)           # store temp in A[1]

   # print_integer(temp);
   move $a0, $s0           # move temp into $a0 for printing
   li $v0, 1               # system call for print integer
   syscall

   # main returns to startup code
   li $v0, 10              # system call for exit
   syscall
```

Exercício 4

Traduza o seguinte programa em C para Assembly MIPS, considerando que a variável i está armazenada no registrador $s0 e o endereço-base do vetor fib está armazenado no registrador $s1.

```
fib[0] = 0      fib[5] = 5
fib[1] = 1      fib[6] = 8
fib[2] = 1      fib[7] = 13
fib[3] = 2      fib[8] = 21
fib[4] = 3      fib[9] = 34
```

```c
#include <stdio.h>

int fib[10];

int main(void) {

    int i = 2;

    fib[0] = 0;
    fib[1] = 1;

    while (i < 10) {
        fib[i] = fib[i - 1] + fib[i - 2];
        i++;
    }

    // Exibir a sequência de Fibonacci
    for (int j = 0; j < 10; j++) {
        printf("fib[%d] = %d\n", j, fib[j]);
    }

    return 0;
}
```

Exercício 4 - Resposta

```
# Variable initialization
li $s0, 2        # i = 2
li $t0, 0        # $t0 = 0
li $t1, 1        # $t1 = 1

sw $t0, ($s1)    # fib[0] = 0
sw $t1, 4($s1)   # fib[1] = 1
```

```
loop:
   bge $s0, 10, end_loop  # break if i >= 10
   sll $t2, $s0, 2        # multiply i by 4 to get the offset in bytes
   addu $t2, $t2, $s1     # add the offset to the base array addr
   lw $t3, -4($t2)        # load fib[i-1] from memory
   lw $t4, -8($t2)        # load fib[i-2] from memory
   addu $t5, $t3, $t4     # add fib[i-1] and fib[i-2]
   sw $t5, ($t2)          # store the sum in fib[i]
   addiu $s0, $s0, 1      # increment i
   j loop                 # jump back to the beginning

end_loop:
```

Exercício 5

Considere vec como um vetor de números naturais, que contém os dígitos (números de 0 a 9) da representação decimal do número natural num = 19865. O primeiro elemento do vetor representa o dígito menos significativo. O programa em C a seguir converte cada um dos elementos do vetor vec para sua representação ASCII e os armazena na cadeia de caracteres chain.

Em cada iteração do laço, o dígito vec[5-1-i] é convertido, em vez do dígito vec[i]. Observe que a conversão para ASCII é realizada adicionando-se 48 ao dígito decimal, ou seja, o código ASCII do caractere '0'.

Observe também que, quando o vetor de caracteres chain é declarado, é reservado espaço para 5+1 elementos, a fim de armazenar o valor sentinela 0, que indica o final da cadeia de caracteres (string).

```c
#include <stdio.h>

int main(void) {
    char chain[6] = {-1, -1, -1, -1, -1, -1};

    unsigned int vec[5] = {5, 6, 8, 9, 1};

    int i = 0;

    while (i < 5) {
        chain[i] = vec[4 - i] + '0';
        i++;
    }

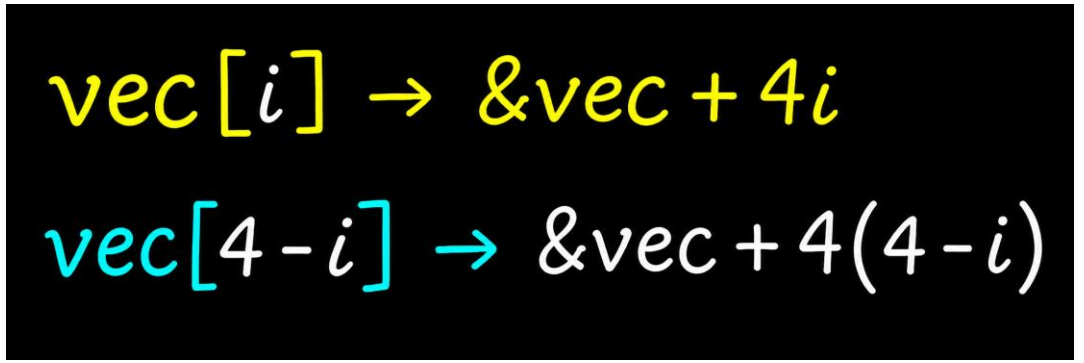
    chain[5] = '\0';

    printf("%s\n", chain);

    return 0;
}
```

Exercício 5 - Resposta

```
.data
chain: .byte -1,-1,-1,-1,-1,-1
vec:   .word 5,6,8,9,1
.text
.globl main
main:
   li $s0,0          # i = 0
while:
   li $t0,5
   bge $s0,$t0,end          # jump to end if i >= 5
   li $t1,4
   sub $t1,$t1,$s0          # 4 - i
   la $s1,vec               # s1 = &vec
   sll $t1,$t1,2            # t1 = (4-i) * 4
   addu $s1,$s1,$t1         # s1 = &vec + (4-i)*4
```



Exercício 5 - Resposta

```
.data
chain: .byte -1,-1,-1,-1,-1,-1
vec:   .word 5,6,8,9,1
.text
.globl main
main:
   li $s0,0          # i = 0
while:
   li $t0,5
   bge $s0,$t0,end          # jump to end if i >= 5
   li $t1,4
   sub $t1,$t1,$s0          # 4 - i
   la $s1,vec               # s1 = &vec
   sll $t1,$t1,2            # t1 = (4-i) * 4
   addu $s1,$s1,$t1         # s1 = &vec + (4-i)*4
   lbu $s2,0($s1)           # s2 = vec[4-i]
   addiu $s2,$s2,48         # s2 = s2 + 48

   la $s3,chain             # s3 = &chain
   addu $s3,$s3,$s0         # s3 = &chain + i
   sb $s2,0($s3)            # chain[i] = vec[4-i] + '0'
   addiu $s0,$s0,1          # i++
   b while

end:
   la $t5,chain             # t5 = &chain
   sb $zero,5($t5)          # char[5] = 0
   li $v0,4                 # print_chain call
   la $a0,chain             # parameter chain
   syscall
   jr $ra
```

Atividade Avaliativa 6

Utilizando o Assembler MIPS (Simulador MARS)

Exercício 1

Desenvolva um programa que solicite ao usuário a digitação de 10 números inteiros.

Para cada número informado, o programa deverá chamar uma subrotina denominada verifica_paridade, responsável por verificar se o número é par ou ímpar.

A subrotina deverá retornar:

0: caso o número seja par.

1: caso o número seja ímpar.

O programa principal deverá apresentar o resultado de cada verificação.

Exercício 2

Desenvolva uma calculadora que apresente o seguinte menu:

- Adição
- Subtração
- Multiplicação
- Divisão
- Sair

O programa deverá solicitar dois números inteiros e executar a operação selecionada pelo usuário.

Cada operação deverá ser implementada em uma subrotina independente:

- soma
- subtracao
- multiplicacao
- divisao

O menu deverá ser apresentado novamente após cada operação, até que o usuário selecione a opção de saída.

Exercício 3

Desenvolva um programa que solicite ao usuário um número inteiro não negativo e calcule seu fatorial.

O cálculo deverá ser realizado por uma subrotina denominada fatorial, que receberá o número como parâmetro e retornará o resultado.

Exemplo:

Entrada: 5

Saída: 120

O programa deverá permitir que o usuário realize vários cálculos, encerrando somente quando informar -1.

Exercício 4

Desenvolva um programa que solicite ao usuário um número inteiro positivo e verifique se ele é primo.

A verificação deverá ser realizada por uma subrotina denominada eh_primo, que receberá o número como parâmetro e retornará:

1: se o número for primo.

0: se o número não for primo.

O programa deverá verificar todos os números entre 2 e 100, apresentando aqueles que são primos.

Exercício 5

Desenvolva um programa que solicite ao usuário 10 números inteiros e armazene-os em um vetor.

Após o preenchimento, o programa deverá identificar o maior e o menor elemento do vetor.

A identificação deverá ser realizada por duas subrotinas:

- encontra_maior: recebe o endereço do vetor e sua quantidade de elementos, retornando o maior valor.
- encontra_menor: recebe o endereço do vetor e sua quantidade de elementos, retornando o menor valor.

Exercício 6

Desenvolva um programa que solicite ao usuário a quantidade de elementos da sequência de Fibonacci que deseja gerar.

O programa deverá armazenar os elementos em um vetor e apresentar a sequência completa.

A geração deverá ser realizada por uma subrotina denominada gera_fibonacci, que receberá:

- O endereço inicial do vetor.
- A quantidade de elementos que deverão ser gerados.

Exemplo:

- Entrada: 8
- Saída: 0, 1, 1, 2, 3, 5, 8, 13

Exercício 7

Desenvolva um programa que solicite ao usuário 15 números inteiros e armazene-os em um vetor.

Em seguida, solicite um número de referência e determine quantas vezes ele aparece no vetor.

A contagem deverá ser realizada por uma subrotina denominada conta_ocorrencias, que receberá:

- O endereço do vetor.
- A quantidade de elementos.
- O número que deverá ser procurado.

A subrotina deverá retornar a quantidade de ocorrências encontradas.

Exercício 8

Desenvolva um programa que solicite ao usuário 10 números inteiros e armazene-os em um vetor.

Após o preenchimento, o programa deverá ordenar os elementos em ordem crescente utilizando o algoritmo Bubble Sort.

A ordenação deverá ser implementada em uma subrotina denominada ordena_vetor, que receberá o endereço do vetor e a quantidade de elementos.

Exemplo:

Entrada:8, 3, 9, 1, 5

Saída:1, 3, 5, 8, 9

Exercício 9

Desenvolva um programa que simule um sistema de autenticação utilizando uma senha numérica previamente definida no código.

O programa deverá solicitar ao usuário que informe uma senha.

O sistema deverá permitir no máximo três tentativas de autenticação.

A validação deverá ser realizada por uma subrotina denominada valida_senha, que receberá a senha informada e retornará:

- 1: senha correta.
- 0: senha incorreta.

Caso a senha esteja correta, o programa deverá apresentar uma mensagem de acesso autorizado.

Caso o usuário erre as três tentativas, deverá ser apresentada uma mensagem de acesso bloqueado.

Exercício 10

Desenvolva um programa que solicite ao usuário 20 números inteiros e armazene-os em um vetor.

Após o preenchimento, o programa deverá apresentar um menu com as seguintes opções:

- Calcular a soma dos elementos.
- Calcular a média aritmética.
- Identificar o maior elemento.
- Identificar o menor elemento.
- Contar os números pares.
- Contar os números ímpares.
- Sair.

Cada operação deverá ser implementada em uma subrotina independente.

O menu deverá continuar sendo apresentado até que o usuário selecione a opção de saída.

Atividade Avaliativa

1. Gerar 1 arquivo “.asm” para cada exercício proposto.
2. Compactar os arquivos em “.zip”.
3. Enviar o “.zip” no link correspondente no NEAD.

OBS: A atividade pode ser feita em grupos com até 3 alunos.